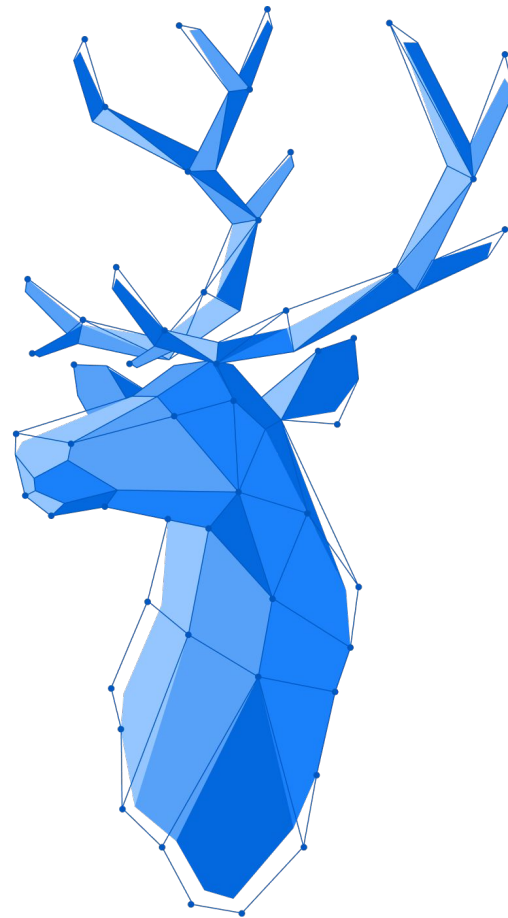




Lazy Loading Data Provider

By Gerald Koch



Lazy Loading Concept

- Lazy loading means just loading things you need right now and other things later when you need them. I.e.: your Grid has 15000 rows, but you see just 50 - so load just 50 and the rest step by step while scrolling.
 - Reduces startup time
 - Content loaded dynamically while scrolling
- Vaadin Components like Grid, TreeGrid, Combobox do already lazy loading between client and server even when you set 1000 items
- To speed up the server side, you can also introduce lazy loading there
- In Vaadin done via data providers
- 2 methods of lazy loading
 - By offset and limit
 - By page

Page vs. Limit and Offset

Page

- Defined by page number and page size
- Supported by Spring Data
- Good when you don't need to know the whole amount of data in advance
- All in one Object
- One method in data provider
 - fetch

Offset and Limit

- Supported by Spring Data and Streams
- Good when you want to know the amount of data beforehand (i.e: when you want to show the number of records)
- Closer to SQL
- 2 methods in data provider
 - fetch
 - count

Data Providers for Components

Base Data Provider

- `DataProvider<T, F> → T = item type, F = filter`

`Grid<T>` does not support in component filters

- `DataProvider<T, Void> → T = item type, no filter`

`TreeGrid<T>`

- `HierarchicalDataProvider<T, ?> → T = item type, does allow but ignores filter`
- Additional methods for parent-child-relationship

`ComboBox<T>`

- `DataProvider<T, String> → T = item type, String filter`

`VirtualList<T>`

- `DataProvider<T, ?> → T = item type, does allow but ignores filter`

But how do I Filter a Grid?

*** Write your own data provider! ***

- When respecting layer isolation, data providers need to be wrapped anyways
- Intercept the path of a dataprovider and add a filter object to your dataprovider which you can forward to the persistence layer, where it will be evaluated
- Create an adapter class for the Vaadin data providers for Grid, ComboBox, etc. in the UI layer
- Create your own `ORDER BY` reflecting classes
- Write converters for sorting and if necessary for paging classes
- Please remember, that the component filter mostly is different from the filter you want to use